

SATYAJIT HALDER

Robotics Software Engineer

☎ +91-6297964646 ✉ satyajithalder806@gmail.com 🔗 linkedin.com/in/hsatyajit25 🐙 github.com/HSatyajit95

Professional Summary

Robotics Software Engineer with **3+ years** building **real-time, multithreaded C++ systems, robot task lifecycle state machines, and fleet-level coordination software** for autonomous robots. Deep expertise in **Modern C++ concurrency, TCP/IP socket communication, behavior trees (BehaviorTree.CPP), and high-frequency task allocation**. Hands-on experience with **ROS 2, Nav2, RabbitMQ, MongoDB, gRPC, REST APIs, Docker, and event-driven backend architectures** through professional and independent **AMR/AGV warehouse automation** projects. MS from **IIT Madras**. Seeking to bring real-time systems depth to large-scale **Fleet Management System (FMS)** development for intralogistics.

Technical Skills

Languages	Modern C++17 (STL, templates, RAII), multithreading & concurrency (mutexes, condition variables, atomics, thread pools, synchronization), Python (scripting & automation, NumPy, OpenCV, FastAPI), MATLAB, C
Real-Time & Control	Deterministic control loops (<1 ms), state machine design, behavior tree architecture (BehaviorTree.CPP), PID / nonlinear control, trajectory planning, fault recovery, emergency stop systems
Robotics & Autonomy	ROS 2 (Humble/Iron), Gazebo, RViz2, Nav2, SLAM Toolbox, A*/Dijkstra/RRT/RRT* path planning, navigation & localization, AMR/AGV fleet coordination, task allocation, multi-robot traffic management concepts, 6-DoF manipulation, DLS inverse kinematics, Ruckig motion planning
Backend & Architecture	Scalable backend services, event-driven architecture, distributed systems design, RabbitMQ (message brokering), MongoDB, REST API development, microservice patterns (Docker Compose)
Communication & Integration	TCP/IP socket programming, robot-to-server protocols, gRPC / Protocol Buffers, CAN bus, OPC UA (familiar), WCS / WES / ERP integration patterns, PLC, conveyor & lift integration concepts (industrial automation & intralogistics)
Computer Vision & AI	Stereo calibration (<50 µm), 3D point clouds (Open3D), depth estimation, YOLOv3, PyTorch, ONNX, CUDA
Dev Tools & Platforms	Linux/Ubuntu, Git (GitFlow, code reviews), CMake, CPack, Docker, gTest/cTest, CI/CD (GitHub Actions, Jenkins – familiar), NSIS (installer packaging), GDB & production debugging, GitHub Copilot / Claude Code, technical documentation
CAD / CAE	SOLIDWORKS, CATIA, CREO, ANSYS
Soft Skills	Technical leadership, cross-functional collaboration, code review, ownership & accountability, goal-oriented execution, mentorship

Projects

Multi-Robot Fleet Management System (FMS) Simulator 2026 – Present

ROS 2 · Gazebo · Nav2 · BehaviorTree.CPP · C++ · gRPC · MongoDB · RabbitMQ · Docker

- Designed and implemented a **multi-robot fleet management system** in simulation coordinating **4 differential-drive AMRs** across a custom Gazebo warehouse environment, replicating real-world FMS architecture.
- Built a **C++ fleet controller** using **BehaviorTree.CPP** for hierarchical task allocation — assigning pick, transport, and drop tasks to robots based on proximity, battery state, and queue priority with **sub-100 ms dispatch latency**.
- Implemented **per-robot state machines** managing full task lifecycles: idle → assigned → navigating → executing → reporting → recovery, with deterministic fault isolation and auto-recovery on Nav2 failures.
- Developed a **gRPC-based fleet server** (Protocol Buffers) for robot-to-server status reporting and command streaming; exposed a **REST API** (Python FastAPI) for external fleet status queries and task injection — mirroring WCS/ERP integration patterns.
- Integrated **RabbitMQ** as a task message broker for decoupled task queuing between the fleet controller and individual robot agents; used **MongoDB** for persistent task logging, robot telemetry, and completion records.
- Wrote **gTest/cTest unit and integration tests** for the fleet scheduler, state machine transitions, and BT node logic; automated with **CMake + CPack** build system and **GitHub Actions CI/CD** pipeline.
- Containerized the entire stack (fleet server, message broker, database, ROS 2 nodes) using **Docker Compose** for reproducible deployment and **field acceptance testing** of the integrated system.

Nav2 Autonomous Navigation with Custom Behavior Tree Plugins 2026

ROS 2 · Nav2 · Gazebo · SLAM Toolbox · BehaviorTree.CPP · C++ · gTest

- Deployed a **full Nav2 autonomous navigation stack** on a simulated differential-drive robot in Gazebo, including SLAM Toolbox for online map building, AMCL localization, and costmap configuration for static and dynamic obstacle layers.
- Developed **custom BehaviorTree.CPP action and condition nodes** as Nav2 plugins — implementing adaptive waypoint following, zone-based speed regulation, and dynamic goal cancellation triggered by occupancy grid updates.
- Tuned **DWB local planner** and **Regulated Pure Pursuit** controller parameters for smooth navigation in narrow-aisle warehouse layouts; implemented custom recovery behaviors (spin, backup, replan) as state machine nodes.
- Built a **Python-based mission executor** using Nav2 Simple Commander API to dispatch multi-waypoint patrol and delivery missions; integrated with ROS 2 action servers for asynchronous goal monitoring and preemption.
- Validated navigation performance with **gTest-based automated tests** replaying rosbag trajectories and asserting localization error bounds; documented all BT node interfaces and planner parameters in a structured technical wiki.

Real-Time Multi-Agent Task Scheduler 2025

C++17 · gRPC · Protocol Buffers · MongoDB · RabbitMQ · Docker · GitHub Actions · gTest

- Built a **deterministic real-time task scheduling engine** in C++17 for coordinating multiple software agents — implementing a **priority-based task queue**, **preemptive state machine dispatcher**, and deadline-aware execution monitor with configurable scheduling policies.
- Designed a **gRPC service layer** (Protocol Buffers schema) for inter-agent communication — enabling agents to register, report status, and receive task assignments with **low-latency RPC calls** (<5 ms round-trip in local testing).
- Integrated **RabbitMQ** for asynchronous task event streaming and **MongoDB** for persistent agent state, task history, and performance metrics; exposed a **REST API** for external task submission and monitoring dashboard.
- Achieved **100% unit test coverage** of scheduling logic and state machine transitions using **gTest**; packaged with **CMake + CPack**, containerized with **Docker**, and automated via **GitHub Actions CI/CD** with build, test, and packaging stages.

Mobile Robot Path Planning & Kinematics Library 2024 – 2025

C++17 · CMake · CPack · gTest · GitHub Actions · Python (pybind11)

- Developed an **open-source C++ path planning library** implementing **A***, **Dijkstra**, **RRT**, and **RRT*** on occupancy grids and continuous configuration spaces, with Python bindings via pybind11 for rapid prototyping.
- Extended existing **DLS-based inverse kinematics solver** (from professional work) into a standalone library supporting both **serial manipulators and differential-drive mobile robot kinematic models** — enabling unified path and motion planning.
- Benchmarked planners across map sizes (100×100 to 2000×2000 grids) using **gTest performance fixtures**; documented results in a structured README with complexity analysis and tuning guidelines. Published on GitHub with **CI/CD via GitHub Actions**.

Work Experience

Transasia Bio-Medicals Ltd.

R&D Engineer — Robotics & Real-Time Systems

April 2024 – Present

Bangalore, India

- **Real-Time Robot Control Framework (C++, Maxon EPOS):** Engineered a **deterministic real-time control framework** for a 6-DoF serial manipulator with Maxon EPOS motor controllers. Designed **TCP/IP and CAN/USB communication layers** for sub-millisecond command execution and high-frequency feedback. Implemented **joint-space control pipelines, multi-axis synchronization, and safety-critical task lifecycle management** including emergency stop, fault detection, and recovery state machines. Leveraged **GitHub Copilot and Claude Code** to accelerate unit test writing and technical documentation.
- **State Machine & Task Lifecycle Architecture:** Designed **hierarchical state machine architectures** governing complex multi-step robot task lifecycles (idle → pre-position → execute → validate → recovery) with deterministic transitions and fault isolation. Structured architecture to align with **behavior tree design principles** for modular, reusable task management compatible with BehaviorTree.CPP patterns.
- **Custom Kinematics & Motion Planning Engine (C++):** Developed **custom DLS-based inverse kinematics** and trajectory planning algorithms in C++, outperforming Pinocchio on a Renesas RZ/V2L SBC. Integrated **Ruckig motion planning library** for jerk-limited trajectory generation; developing an offline trajectory planner to eliminate runtime dependencies.
- **Modular C++ Architecture, Testing & Documentation:** Prototyped in Python, then **ported and optimized to C++** for constrained hardware deployment. Built a **CMake-based testing and validation suite** for system bring-up, calibration, and multi-axis coordination — functioning as unit and **field acceptance test harness**. Maintained structured **technical documentation** of all algorithms, interfaces, and system changes.
- **Vision-Guided Manipulation & 3D Perception:** Designed end-to-end pipeline: stereo camera calibration (<50 μm), eye-to-hand calibration, Unimatch-based 3D point cloud construction, novel **surface normal estimation** for object pose detection, and autonomous navigation to target for precise insertion.

ICSR, IIT Madras

Project Associate — Robotics & Computer Vision

August 2023 – March 2024

Chennai, Tamil Nadu

- **Project: Robotic Ingot Handling System for Hazardous Environment (BARC).**
- Developed **YOLOv3-based 3D localization** using Intel RealSense D455 — mapping bounding boxes to depth frames for ingot pose estimation via Open3D point cloud reconstruction.
- Designed **patentable form-closure gripper** and integrated detection, EoT crane control, and Python GUI into a complete demonstration system validated at IIT Madras.

Research Experience

MS Research — IIT Madras

Research Scholar, Dept. of Mechanical Engineering

January 2019 – March 2024

Chennai, Tamil Nadu

- **Biomechanical Motion Optimization:** Formulated optimization problems predicting sit-to-stand trajectories, minimizing integrated joint torques across five actuated joints. Developed simulation framework validating assistive motion models.
- **Sit-to-Stand Assistive Device (Patented, 2024):** Designed two device variants using four-bar linkage synthesis and double-parallelagram scissor mechanisms with linear actuators. Validated on volunteers — **45% reduction in ground reaction force**.

Education

Indian Institute of Technology, Madras

MS in Mechanical Engineering (Robotics & Dynamics)

2024

Chennai, Tamil Nadu

Government College of Engineering and Textile Technology, Berhampore

B.Tech in Mechanical Engineering

2018

Berhampore, West Bengal

Achievements & Certifications

- **Patent (2024):** Sit-to-stand assistive device — IIT Madras.
- **First Prize:** Students Mechanism and Design Competition (SMDC), IIT Mandi, 2019.
- **Machine Learning A-Z** — Udemy, 2023.
- **Deep Learning A-Z** — Udemy, 2023.
- **Data Structures & Algorithms (C/C++)** — Udemy, 2022.
- **Python for Data Science** — Great Learning, 2022.

Publications

- “Design and Development of a Mobile Sit-to-Stand Assistive Device,” *Journal of The Institution of Engineers (India): Series C*, Vol. 105, pp. 299–312, 2024.
- “Design and Development of a Sit-to-Stand Assistive Device,” *Machines, Mechanism and Robotics*, Proceedings of iNaCoMM 2019.